

Module 1

Algorithm Fundamentals

Module Overview

- **Algorithms** - Definition, Properties, Why Study?
- **Problem Solving Fundamentals** - Framework & Approach
- **Important Problem Types** - Classification & Examples
- **Analysis Framework** - Correctness, Time, Space
- **Asymptotic Notations** - Big-O, Big-Omega, Big-Theta
- **Efficiency Classes** - Comparison & Growth Rates
- **Mathematical Analysis** - Non-Recursive & Recursive

What is an Algorithm?

Definition: An algorithm is a finite set of well-defined instructions to solve a problem or perform a computation.

Key Properties:

- **Input:** Zero or more inputs
- **Output:** One or more outputs
- **Definiteness:** Each step is clear and unambiguous
- **Finiteness:** Must terminate in finite time
- **Effectiveness:** Each operation must be basic and feasible

Algorithmic Problem Solving Framework

7-Step Problem Solving Process:

1 **Understand** the problem completely

2 **Ascertain** capabilities & limitations

3 **Choose** among possible approaches

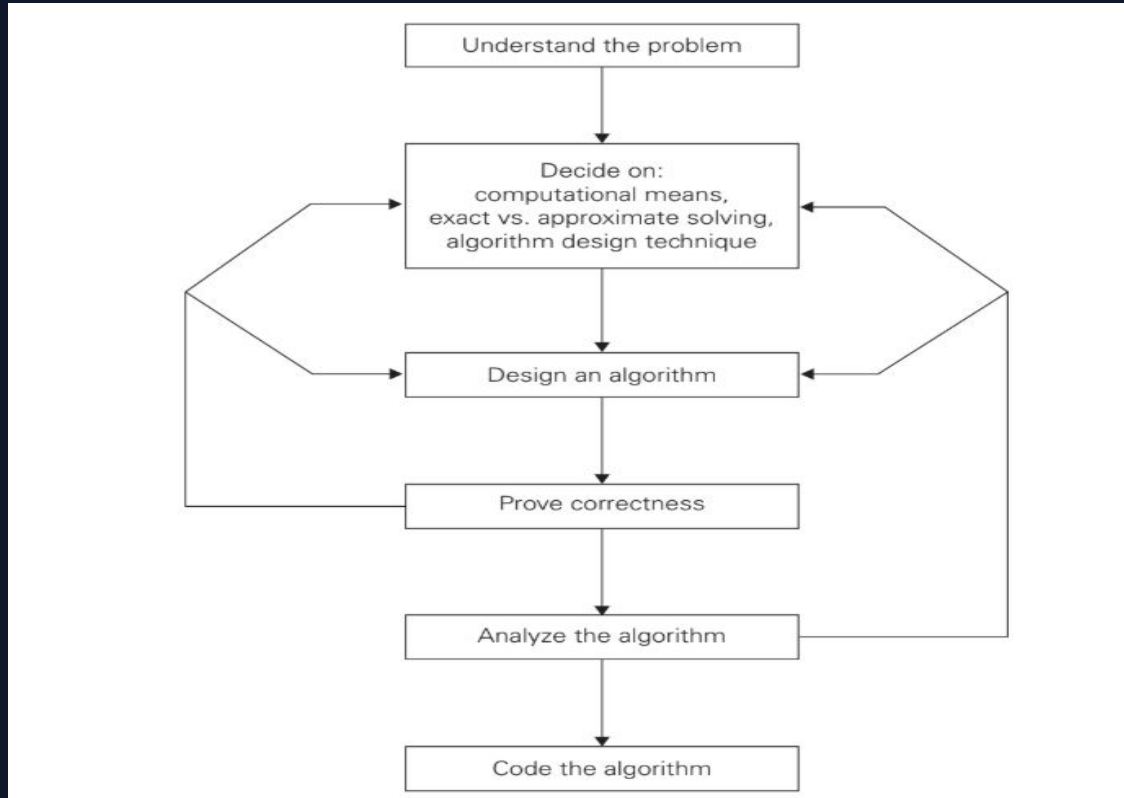
4 **Describe** the algorithm precisely

5 **Prove** algorithm correctness

6 **Analyze** time & space complexity

7 **Implement** efficiently & test

Algorithmic Problem Solving Framework



Important Problem Types

1. **Sorting** - Arrange elements in order (Quick Sort, Merge Sort)
2. **Searching** - Find element in collection (Binary Search, Linear)
3. **String Processing** - Pattern matching, parsing (KMP, Boyer-Moore)
4. **Graph Problems** - Traversal, paths (DFS, BFS, Dijkstra)
5. **Combinatorial** - Permutations, subsets (Backtracking)
6. **Geometric** - Coordinate/shape problems (Convex Hull)
7. **Numerical** - Math computations (GCD, Primality)

Analysis Framework - Pillars

Correctness

Does algorithm produce correct output for all valid inputs?

Time Efficiency

How fast? Count operations relative to input size

Space Efficiency

Memory required? Extra space beyond input

Best, Worst, and Average Case

Worst Case $W(n)$: Maximum over all inputs of size n

Best Case $B(n)$: Minimum over all inputs of size n

Average Case $A(n)$: Expected operations for typical input

Example: Linear Search - Best $O(1)$, Worst $O(n)$, Average $O(n/2)$

Asymptotic Notations - Overview

Mathematical tools to describe algorithm efficiency as input size approaches infinity

Big-O (O)

Upper Bound
At most $f(n)$

Big-Omega (Ω)

Lower Bound
At least $f(n)$

Big-Theta (Θ)

Tight Bound
Exactly $f(n)$

Big-O Notation (Upper Bound)

$f(n)$ is $O(g(n))$ if $\exists$ positive constants c, n_0 such that:
 $f(n) \leq c \cdot g(n)$ for all $n > n_0$

Example Calculation:

$f(n) = 2n^2 + 3n + 5$
For $c = 3, n_0 = 1$:
 $f(n) \leq 3n^2$ (for all $n \geq 1$)

Therefore: $f(n)$ is $O(n^2)$

Worst-case scenario. Most commonly used notation.

Big-Omega Notation (Lower Bound)

$f(n)$ is $\Omega(g(n))$ if $\exists$ positive constants c, n_0 such that:
 $f(n) \geq c \cdot g(n)$ for all $n > n_0$

Example Calculation:

$$f(n) = 2n^2 + 3n + 5$$

For $c = 1, n_0 = 1$:

$$f(n) \geq n^2 \text{ (for all } n \geq 1)$$

Therefore: $f(n)$ is $\Omega(n^2)$

Best-case scenario. Lower bound on growth rate.

Big-Theta Notation (Tight Bound)

$f(n)$ is $\Theta(g(n))$ if $\exists$ positive constants c_1, c_2, n_0 such that:
 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n > n_0$

Example Calculation:

$$f(n) = 2n^2 + 3n + 5$$

For $c_1 = 1, c_2 = 3, n_0 = 1$:

$$n^2 \leq f(n) \leq 3n^2 \text{ (for all } n \geq 1)$$

Therefore: $f(n)$ is $\Theta(n^2)$

Tight bound - best and worst case have same order.

Asymptotic Notations Comparison

Notation	Meaning	Analogy	Symbol
Big-O: $O(g)$	$\leq$ (Upper bound)	$\leq$ (less than or equal)	O
Big-Omega: $\Omega(g)$	$\geq$ (Lower bound)	$\geq$ (greater than or equal)	Ω
Big-Theta: $\Theta(g)$	$=$ (Tight bound)	$=$ (equal to)	Θ

Basic Efficiency Classes

Class	Name	Example
$O(1)$	Constant	Array access by index
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Linear search
$O(n \log n)$	Linearithmic	Merge sort, Quick sort
$O(n^2)$	Quadratic	Bubble sort, Nested loops
$O(n^3)$	Cubic	Matrix multiplication
$O(2^n)$	Exponential	Recursive Fibonacci
$O(n!)$	Factorial	All permutations

Growth Rate Comparison

For $n = 100,000$ operations:

Class	$n=100$	$n=1000$	$n=10000$
$O(1)$	1	1	1
$O(\log n)$	6.6	9.9	13.2
$O(n)$	100	1,000	10,000
$O(n \log n)$	664	9,965	132,877
$O(n^2)$	10,000	1M	100M

Analysis of Non-Recursive Algorithms

5-Step Procedure:

- 1 **Decide** input size parameter (n)
- 2 **Identify** basic operation (innermost loop)
- 3 **check** how many times the basic operation is executed, then find it depends upon the size n .
- 4 **Set up** summation for $C(n)$
- 5 **Simplify** using standard formulas

Non-Recursive Example: Linear Search

```
Algorithm LinearSearch(A[0..n-1], K) for i = 0 to n-1 do if A[i] = K return i return -1
```

Input size: n (array length)

Basic operation: Comparison $A[i] = K$

Best case: $B(n) = 1$ (found at first position)

Worst case: $W(n) = n$ (found at last or not found)

Analysis of Recursive Algorithms

5-Step Procedure:

- 1 **Decide** input size parameter (n)
- 2 **Identify** basic operation (innermost loop)
- 3 **check** how many times the basic operation is executed, then find it depends upon the size n .
- 4 **Set up** recurrence relation
- 5 **Solve** the recurrence or atleast determine the order of growth.

Thank You